

Face- und Fiducialtracking

Worum geht es in diesem Blatt?

In diesem Blatt arbeiten Sie mit kamerabasierter Erkennung in vvvv. Sie lassen Gesichter in einem Video und im Kamera-Livestream finden und verfolgen anschließend Fiducial-Marker – gedruckte Muster, die sich zuverlässiger erkennen lassen als ein Gesicht. Beide Verfahren liefern Positionen, auf die eine Anwendung reagieren kann.

Was erwarten wir uns von der Praktikumsdokumentation/Protokoll?

- Beantworten der gestellten Fragen in einem Protokoll (Moodle-Befragung)
- Zusätzlich machen Sie bitte Screenshots ihrer Patches/Programme sowie Fotos, die Ihre Lösungen in „Aktion“ zeigen (vvvv-Screenshots können Sie direkt in „vvvv“ komfortabel über die Tastenkombinationen „**STRG + 1**“ bzw. „**STRG + 2**“ erstellen); abspeichern der Patches nicht vergessen!
- **Eigene Interpretation** von Erkenntnissen des Experiments, Auffälligkeiten
- Welche **Probleme** sind Ihnen begegnet und wie haben Sie diese **gelöst**?

Wie?

- **Beantworten Sie zur Abgabe den Fragebogen „Moodle-Abgabe Blatt 6“** in Moodle (Kursraum Technik der Mensch-Maschine Interaktion).
- **Kopieren Sie dazu Ihre** Antworten (Protokoll ggfs. in der Praktikumsstunde direkt im Angabe-Worddokument oder separatem Textdokument erstellen), **Screenshots, Fotos, etc. in das Antwortfeld.**
- **vvvv-Patches abspeichern und als Anhäng mitabgeben!**
- **ACHTUNG: Chromium-basierte Browser scheinen keine Bilder kopieren zu können, nutzen Sie in dem Fall einen anderen Browser, z.B. Firefox**

Abgabe?

- **Abgabe innerhalb von 48 Stunden** nach Ihrem Praktikumstermin, in Moodle

Für dieses Aufgabenblatt ist zwingend folgende Datei-Ordnerstruktur zu verwenden (diese finden Sie als Kopie am Laufwerk „P:“ (Datentausch)). KOPIEREN SIE BITTE DEN GESAMTEN ORDNER AUF DEN DESKTOP ODER IHR PERSÖNLICHES VERZEICHNIS. Die .v4p-Patches sind in den korrekten Ordnern abzulegen – sonst funktionieren die Programme evtl. nicht!

```
TMIP_PR5
|_Facedetection
|_assets (Haarcascades.wmv; "Cascade Training Files")
|_Fiducialtracking
|_assets (Fiducialtracking.wmv)
|_FiducialTrackingv1_Parametrisierung.v4p
```

In diesem Aufgabenblatt dreht sich alles um visuelle Datenverarbeitung. Wir verwenden beispielsweise Facetracking/-detection, um Merkmale des menschlichen Gesichts in einem Video bzw. Kamera-Livestream zu erkennen und verwenden sogenannte „*Fiducials*“ zur einfachen Demonstration von marker-basiertem Tracking. „*Fiducial*“-Maker nutzt man häufig in „*Augmented Reality*“-Applikationen.

1) Gesichtserkennung in einem Video bzw. Kamera-Livestream

In dieser ersten Aufgabe verwenden wir „vvvv“ um Position und Größe eines Gesichts in einem Video bzw. in einem Livestream einer Webcam zu erkennen. Wir beschäftigen uns hier mit der Gesichtsdetektion, d.h. dem Finden und Kennzeichnen von Gesichtern oder Gesichtsfutures in einem Bild. Über entsprechende Analysekomponenten wäre es sogar möglich, das Alter oder Geschlecht aus dem Video-/Kamerabild zu extrahieren oder auch mehrere Personen in einem Video zu unterscheiden. Facetracking bzw. -detection als „Eingabegerät“ gehört zur Klasse der natürlichen Benutzerschnittstellen („Natural User Interfaces“), da keine zusätzlichen Gegenstände zur Interaktion benötigt werden. Für die Beispiele dieses Aufgabenblattes verwenden wir die Sony PlayStation Eye (PS3)-Kamera, da diese Bilder mit bis zu 120 Hz liefert und sehr günstig zu erwerben ist (ca. 8 Euro bei Amazon). **Hinweis:** Falls Sie diese Kamera in eigenen Projekten verwenden möchten, müssen Sie auf Ihrem Rechner zunächst den „*CI-Eye Platform Driver*“ der Firma „*Code Laboratories*“ (<https://codelaboratories.com/products/eye/driver>) installieren (für US-\$ 2,99).

In diesem Aufgabenblatt verwenden wir drei sogenannte „*Haar-Feature*“-Trainingsdateien (Infos dazu: https://en.wikipedia.org/wiki/Haar-like_features), um a) den Kopf- und Schulterbereich in einem Bild zu finden, b) einfache Gesichtsdetektion (frontal) zu implementieren und c) die Augen bzw. das Augenpaar in einem Bild zu identifizieren (herunterladen und abspeichern passiert unter Punkt 3 auf Seite 5):

Cascades			
Description & References	Author(s) / Licence	Version	Haar cascade file
Frontal Face stump 24x24, 20x20gentle, 20x20tree	Rainer Lienhart	1.0	frontalFace10.zip
Eyes 22x5 Eye pair detector computed with 7000 positive samples Reference paper	Modesto Castrillón Santana Conditions of use	1.0	Eyes22x5.zip
Head and shoulders 22x20 Head and shoulders classifier Reference paper	Modesto Castrillón Santana Conditions of use	1.0	HS22x20.zip

Schritt 1: Die zentrale „vvvv“-Komponente für die Detektion von Gesichtern (oder anderer Features) in einem Bild oder Video ist der „DetectObject (FreeFrame DShow9)“-Node. Diesem Node übergibt man ein Bild, eine Videodatei, oder einen Kamera-Livestream (Eingangs-Pin „Video Input“). Über den Eingangs-Pin „Training File“ legt man fest, welches Feature bzw. welche Region man in einem Bild erkennen will (z. B. Gesicht, Augen, Mundpartie, etc.). Die weiteren Eingabeparameter („Min Neighbors“, „Scale Cascade“, „Min Facesize“, „Lifetime“) dienen zum Feintuning und werden wir uns weiter unten genauer ansehen.

Vorgangsweise

- Neuen, leeren Patch anlegen
- Nun bauen wir nach und nach das Programm (d.h. den „Patch“), wie nachfolgend dargestellt, auf:
- **Hinweis:** Die Nummernangaben in Klammern entsprechen den jeweiligen Zahlen in den Kreisen der Patch-Screenshots!



(1) Einfügen eines „DetectObject (FreeFrame DShow9)“-Nodes in den Patch

(1) Einfügen eines „DetectObject (FreeFrame DShow9)“-Nodes in den Patch

- Sehen Sie sich dazu den Hilfe-Patch zu „DetectObject“ an (F1-Taste)
- Unter [http://vvvv.org/documentation/detectobject-\(freeframe-dshow9\)](http://vvvv.org/documentation/detectobject-(freeframe-dshow9)) finden Sie die Dokumentation dieses Patches im Internet. Sehen Sie sich auch hier die Funktionalität an

(2) Einfügen eines „IOBox (Value Advanced)“-Nodes, Datentyp „Boolean“, Behavior „Toggle“. Über diesen „Schalter“ können Sie später festlegen, ob der „Tracker“ des identifizierten Objektes im Bild bzw. Video (als rotes Rechteck) angezeigt wird.

(3) Einfügen einer „IOBox (String)“ und verbinden mit dem 3. Eingangs-Pin des „DetectObject (FreeFrame DShow9)“-Nodes. Dieser Node wird verwendet, um (etwas später) eine bestimmte „Trainingsdatei für Haar Cascades“ zu Laden – in unserem Fall für Kopf-/Schulterererkennung, Gesichtserkennung bzw. Augenerkennung. Als Zeichenkette (String) wird der Node automatisch mit dem vollständigen Dateipfad des Verzeichnisses, in dem der aktuelle vvvv-Patch liegt, befüllt (z.B. „C:\Dokumente und Einstellungen\...\Desktop\TMIP5\“. **Wir können hier auch relative Dateipfade eingeben.** (Links neben dem String-Node wird weiters eine Schaltfläche mit dem Text „open“ angezeigt – mit einem Klick darauf öffnet sich der Dateieexplorer - brauchen wir allerdings nicht...). Lassen Sie den Inhalt (Zeichenkette) dieses Nodes zunächst einmal „links liegen“.

(4) Einfügen einer „IOBox (Value Advanced)“, Datentyp „Real“, Wert „20.0“ und verbinden mit dem Eingangs-Pin „Min Face Size“ des „DetectObject (FreeFrame DShow9)“-Nodes.

(5) Einfügen eines „IOBox (Value Advanced)“-Nodes, Datentyp „Boolean“, Behavior „Toggle“. Über diesen „Schalter“ starten wir später die Objekterkennung.

(6) Wir fügen einen „Count(Value)“-Node ein und verbinden diesen mit dem Ausgangs-Pin „Object ID“ des „DetectObject (FreeFrame DShow9)“-Nodes. Der Count-Node ist sehr oft nützlich: Er gibt die Anzahl der „Slices“ in einem „Spread“ zurück und entspricht in Java dem .length-Operator von Arrays. In unserem Fall hier wird damit gezählt/angezeigt, wie viele Personen bzw. Objekte in einem Bild bzw. Video erkannt wurden. Weiters fügen wir einen „IOBox (Value Advanced)“-Node ein und stellen dessen Schriftgröße (Parameter „Size“) im „Herrn Inspektor“ auf „20,0“ px – in diesem Feld/Node wird dann angezeigt, wie viele Objekte der „DetectObject (FreeFrame DShow9)“-Nodes erkannt hat.

(7) Einfügen eines „VideoTexture (EX9.Texture VMR9)“-Nodes und Anschluss an den ersten Ausgangs-Pin des „DetectObject (FreeFrame DShow9)“-Nodes. Dieser Node sorgt dafür, dass später ein Video im Renderer-Fenster abgespielt wird.

(8) Ein „Quad (DX9)“-Node an dessen Eingang „Texture“ der Node (7) „VideoTexture (EX9.Texture VMR9)“ angeschlossen wird

(9) Einfügen eines „Group (EX9)“-Nodes und verbinden wie in der Skizze oben dargestellt

(10) Fügen Sie nun noch einen „Renderer (EX9)“-Node ein, um die Bild-/Videoquelle, die wir nachfolgend an den „DetectObject (FreeFrame DShow9)“-Node übergeben, anzuzeigen

(11) Der „Scale (Transform Vector)“-Node wird verwendet, um die Größe des Bildes bzw. Videos im Renderer anzupassen bzw. zu skalieren. Stellen Sie den Faktor in (12) mit einer „IOBox (Value Advanced)“ zunächst auf „2.0“ ein.

Damit ist die „Logik“ für das Objekttracking fertig und wir wenden uns nun der Eingabe von Bild, Videodatei bzw. Kamera-Livestream zu. Speichern Sie Ihren Patch als „Facedetectionv1.v4p“ ab.

Schritt 2: Wir möchten zunächst eine voraufgezeichnete Videodatei verwenden, um darin Features zu erkennen (später schließen wir eine Webcam an und nutzen den Livestream der Kamera). Zum Öffnen und Wiedergeben von Videos verwenden wir den „FileStream (DShow9)“-Node. Dieser Node kann universell zur Wiedergabe von Audio-, Video- oder Midi-Dateien verwendet werden:

zur Wiedergabe von Audio-, Video- oder Midi-Dateien verwendet we



Vorgangsweise

- Verschieben Sie den bisherigen Teil des Patches (rund um den „DetectObject“-Node) nach unten, um „oben“ Platz für den „FileStream“-Node zu bekommen.
- Nun erweitern wir den Patch nach und nach wie in der Abbildung auf der nächsten Seite dargestellt:

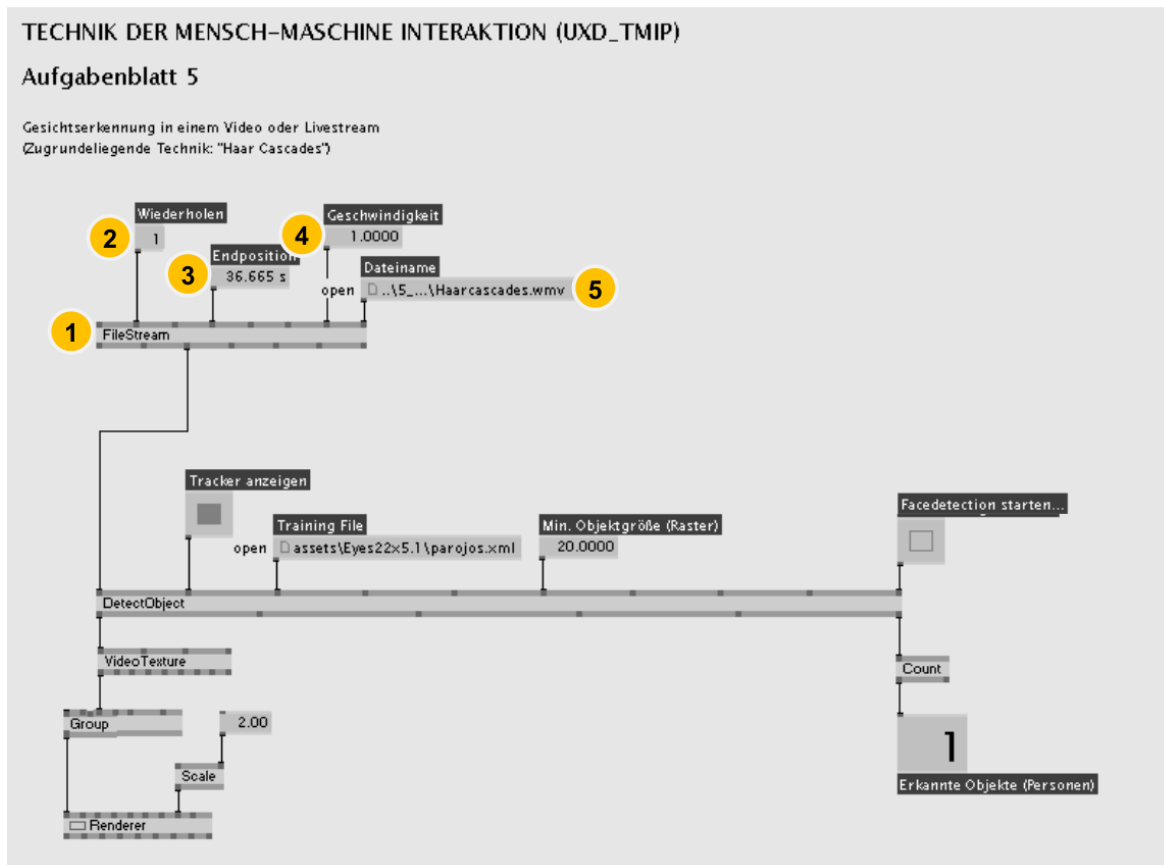
(1) Einfügen eines „FileStream (DShow9)“-Nodes in den Patch und Konfiguration wie folgt:

(2) Ein „IOBox (Value Advanced)“-Node dessen Wert auf „1“ gesetzt wird, um anzugeben, dass die Videoquelle immer wieder wiederholt werden soll (=„loop“).

(3) Ein weiterer „IOBox (Value Advanced)“-Node, mit dem wir festlegen können, ob die gesamte Videodatei oder nur ein Ausschnitt davon wiederholt werden soll. Das Video das wir zunächst für die Objekterkennung verwenden, hat eine Länge von „36,665“ Sekunden. Wir geben hier diesen Wert an (Video wird vollständig wiedergegeben und wiederholt) und verbinden den Node mit dem Eingangs-Pin „Loop End Time“ des „FileStream (DShow9)“-Nodes.

(4) Mit dieser „IOBox (Value Advanced)“ (angeschlossen an den Eingangs-Pin „Speed“ des „FileStream (DShow9)“-Nodes) können wir die Abspielgeschwindigkeit des Videos ändern. Wir stellen den Parameter auf „1.0“ ein.

(5) Einfügen eines „IOBox (String)“-Nodes und verbinden mit dem letzten Eingangs-Pin „Filename“ des „FileStream (DShow9)“-Nodes. Mit diesem Node laden wir ein Video, das wir für die Objekterkennung verwenden möchten. Klicken Sie mit der rechten Maustaste direkt auf den Node – ein Fenster zur Auswahl eines Videos öffnet sich. Öffnen Sie den Ordner „assets“ im Projektverzeichnis von „Facedetection“ und wählen dort die Datei „Haarcascades.wmv“ aus.



Damit ist auch dieser Teil fertig und das Video sollte bereits im Renderer-Fenster abgespielt werden...

Schritt 3: Zu guter Letzt müssen wir uns nun noch die Trainingsdateien für das Objekttracking besorgen und an den „DetectObject (FreeFrame DShow9)“-Node übergeben.

Vorgangsweise

- Öffnen Sie die Webseite <http://alereimondo.no-ip.org/OpenCV/34> und laden Sie (mindestens) folgende Trainingsdateien („*Haarcascade files*“) herunter (siehe dazu auch Seite 2):
- „*Eyes22x5.zip*“ (Trainingsdateien für Augenerkennung)
- „*FrontalFace10.zip*“ (Trainingsdateien für Gesichtserkennung, frontal)
- „*HS22x20.zip*“ (Trainingsdateien für Tracking von Kopf und Schulterbereich)
- „*Mouth25x15.zip*“ (Trainingsdateien für die Erkennung des Mundbereiches)
- Extrahieren Sie die ZIP-Archive und kopieren Sie die erhaltenen Dateien (parajos.xml, haarcascade_frontalface_alt.xml, HS.xml, Mouth.xml) in den Ordner „assets“ ihres Projektverzeichnis „Facedetection“.
- Klicken Sie nun mit der rechten Maustaste auf den mit „Training File“ bezeichneten Node („IOBox (String)“, Nummer (3) oben bei Schritt 1) und wählen „**parajos.xml**“ im „Assets“-Ordner aus.
- Speichern Sie Ihren Patch als „Facedetectionv2.v4p“ ab.
- Aktivieren Sie „Tracker anzeigen“ (Nummer (2)) sowie „Facedetection starten...“ (Nummer (5))

...im Renderer-Fenster sollten nun die Augen „getrackt“ werden.

Frage 1: Was stellen Sie beim Betrachten des Videos (hinsichtlich dem Tracking) fest?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 2: Was bedeutet es, wenn der rechteckige, rote Bereich aus dem Bild verschwindet?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 3: Was stellen Sie fest, wenn Sie den Wert des Parameters „Min Face Size“ ((Punkt (3) in der Patch-Skizze zu Schritt 1 oben, bezeichnet als „Min. Objektgröße (Raster)“ von „20.0“ auf andere Werte ändern? Was könnte der Grund für das festgestellte Verhalten sein? Probieren Sie zumindest folgende Werte aus: a) „40.0“, b) „100.0“, c) „10.0“, d) „2.0“.

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!



Hinweis zum Node „DetectObject“: Zusätzlich zur oben angegebenen Parametrisierung haben die 3 Attribute „Min Neighbors“, „Scale Cascade“ sowie „Lifetime“ einen Einfluss auf die Trackingqualität.

Ein **guter Standardwert ist „1.0“ für alle 3 Parameter**, insbesondere mit Variation von „Scale Cascade“ kann die Trackingqualität – je nach Bildquelle (UmgebungsHELLigkeit, etc.) – signifikant verbessert werden!

Sehen Sie sich nun noch die anderen Tracking-Varianten an. Öffnen Sie dazu als „Training File“ zunächst die Datei **„haarcascade_frontalface_alt.xml“**, dann die **„HS.xml“** und schließlich **„Mouth.xml“**.

Frage 4: Was stellen Sie fest? Was sind „gute“ Werte für die Objektgröße (Raster) für das Kopf-/Schultertracking bzw. Gesichtstracking? Begründen Sie Ihre Wahl für die beiden Werte!

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Schritt 4: Als nächstes verbinden wir „vvv“ mit einer Webcam und verwenden den Kamera-Livestream um daraus Objekte (wiederum a) Augen, b) Kopf+Schulter, c) Gesicht) oder d) Mund zu erkennen und zu „tracken“.

Vorgangsweise

- Wir beginnen mit dem bereits geöffneten Patch und sichern diesen als „FacedetectionV3.v4p“.
- Nun erweitern wir diesen Patch wie folgt:

(1) Einfügen eines „VideoIn (DShow9)“-Nodes in den Patch und Konfiguration wie folgt:

(2) Eine „IOBox (Value Advanced)“, Behavior: „Toggle“, verbunden mit Eingangs-Pin 1 des „VideoIn (DShow9)“-Nodes verwenden zum Aktivieren/Deaktivieren dieses Nodes und damit der angeschlossenen Kamera

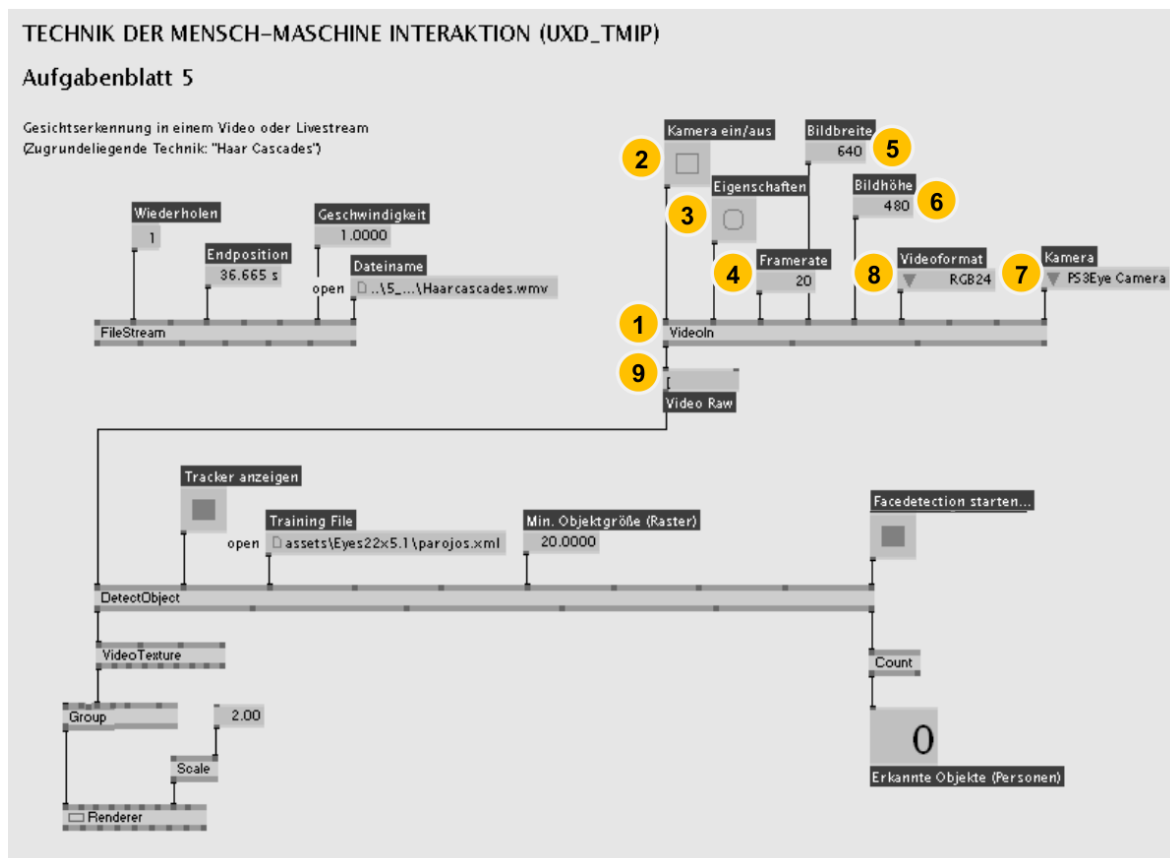
(3) Ein weiterer „IOBox (Value Advanced)“-Node wird mit 2. Eingangs-Pin (benannt als „Property Page“) von „VideoIn (DShow9)“ verbunden, Behavior: „Bang“. Durch Rechtsklick auf diesen Node öffnet sich die Eigenschaften-Seite der jeweils aktiven Kamera – probieren Sie das ruhig aus!

(4) bis (6) sind jeweils Nodes vom Typ „IOBox (Value Advanced)“ mit denen die Bildwiederholrate der Kamera (auch „Framerate“ genannt), die Bildbreite sowie die Bildhöhe festgelegt werden. Stellen Sie die Werte (wie in der Skizze unten ersichtlich) auf 20, 640 bzw. 480 ein.

(7) ist eine „IOBox (Enumerations)“, die mit dem letzten Eingangs-Pin des „VideoIn (DShow9)“-Nodes verbunden wird. Über das Dropdown-Menü (Klick mit rechter Maustaste auf den Pfeil nach unten) kann die gewünschte Kamera ausgewählt werden. Wir verwenden die „PS3Eye Camera“.

(8) ist ebenfalls ein Node vom Typ „IOBox (Enumerations)“, die an den mit „Video Format“ bezeichneten Eingangs-Pin des „VideoIn (DShow9)“-Nodes angehängt wird. Wir stellen als Videoformat „RGB24“ ein.

(9) Fügen Sie zum Schluss einen Node vom Typ „IOBox (Node)“ ein und verbinden Sie den Node mit dem 1. Ausgangs-Pin des „VideoIn (DShow9)“-Nodes sowie mit dem 1. Eingangs-Pin des eines „DetectObject (FreeFrame DShow9)“-Nodes.



Damit ist die Konfiguration bzw. Integration der Webcam komplett. Speichern Sie den Patch ab, aktivieren Sie zuerst die Kamera und danach die Gesichtserkennung (Facedetection starten...).

Frage 5: Wie gut funktioniert das Tracking mit dem Live-Bild im Vgl. zum Video von oben? Verwenden Sie auch hier die Trainingsdateien für Augentracking, Kopf-/Schultertracking, Kopftracking bzw. Mundtracking. Welche Werte sind nun „optimal“ für die „Min. Objektgröße (Raster)“ der 4 Trackingvarianten?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 6: Experimentieren Sie nun mit mehreren Personen. Wie viele Personen schafft der „DetectObject“-Node gleichzeitig? Überwachen Sie einerseits den Zähler mit der Bezeichnung „Erkannte Objekte (Personen)“ sowie andererseits die Anzahl der Rechtecke im Videobild. Was fällt Ihnen dabei sonst noch auf?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

2) Tracking von „Fiducial“-Markern

„Fiducial“-Marker sind bekannte Vertreter des sog. markerbasierten Trackings. Die organisch wirkenden Muster zeichnen sich dadurch aus, dass sie über eine einfache Webcam erfasst und verarbeitet werden können. „Fiducial“-Marker können ausgedruckt und auf jegliche Oberfläche angebracht werden. Bei interaktiven Anwendungen werden die Marker oft auf der Unterseite von Objekten angebracht (und bei interaktiven Tischen von unten abgefilmt) – und sind dadurch für den Benutzer nicht sichtbar.



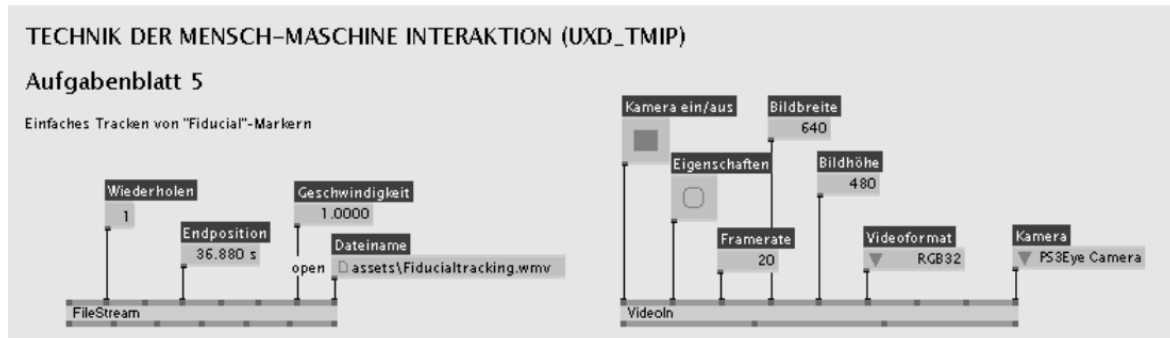
Beispiele für „Fiducial“-Marker; Marker im Einsatz (rechts)

„Fiducial“-Marker ermöglichen es, beliebige Objekte sehr einfach und zuverlässig in einem Kamera-Livestream zu identifizieren. Schon mehrere solcher Marker erlauben es, erste zusammenhängende Interaktionen zu entwerfen – z. B. Verknüpfung unterschiedlicher Objekte (Größe, Distanz zueinander, etc.). Damit ist es sehr einfach möglich, „leblose“ Gegenstände im Kamerabild (und dann in einer Applikation) zum Leben zu erwecken...

Schritt 1: Eine Möglichkeit um beliebige Objekte in einem Live-Kamerabild zu tracken ist die Verwendung von „Fiducial“-Tracking. Die zentrale „vvvv“-Komponente um Fiducial-Marker einzulesen und zu verarbeiten ist der „FiducialTracker (FreeFrame DShow9)“-Node. Dieser Node erkennt „Fiducial“-Marker im übergebenen Kamerabild (bzw. auch Video) und liefert für den Marker die Position (x/y; Ausgabe-Pins „X“ bzw. „Y“) im Bild bzw. den Rotationswinkel (Ausgabe-Pin „Orientation“). Wir möchten hier die Mächtigkeit des Konzepts durch eine einfache Anwendung demonstrieren: Ein Kamera-Livebild bzw. Video wird im Renderer (als Hintergrund) angezeigt, Marker im Bild/Video werden durch den der „FiducialTracker (FreeFrame DShow9)“-Node erkannt und Kreissegmente dem Kamera-/Videobild überlagert...

Vorgangsweise

- Wir beginnen mit dem Patch „Facedetectionv3.v4p“ von vorn und lassen nur die beiden Komponenten „FileStream“ und „VideoIn“ mit sämtlichen Konfigurationsparametern stehen – den Rest (d.h. alles „unterhalb“ des „VideoIn“-Nodes inkl. dem Node mit der Bezeichnung „Video Raw“) entfernen wir aus dem Patch. Das Ergebnis sollte in etwa so aussehen; speichern Sie das Zwischenergebnis als Datei „FiducialTrackingV1.v4p“ (in den Dateordner „Fiducialtracking“) ab.

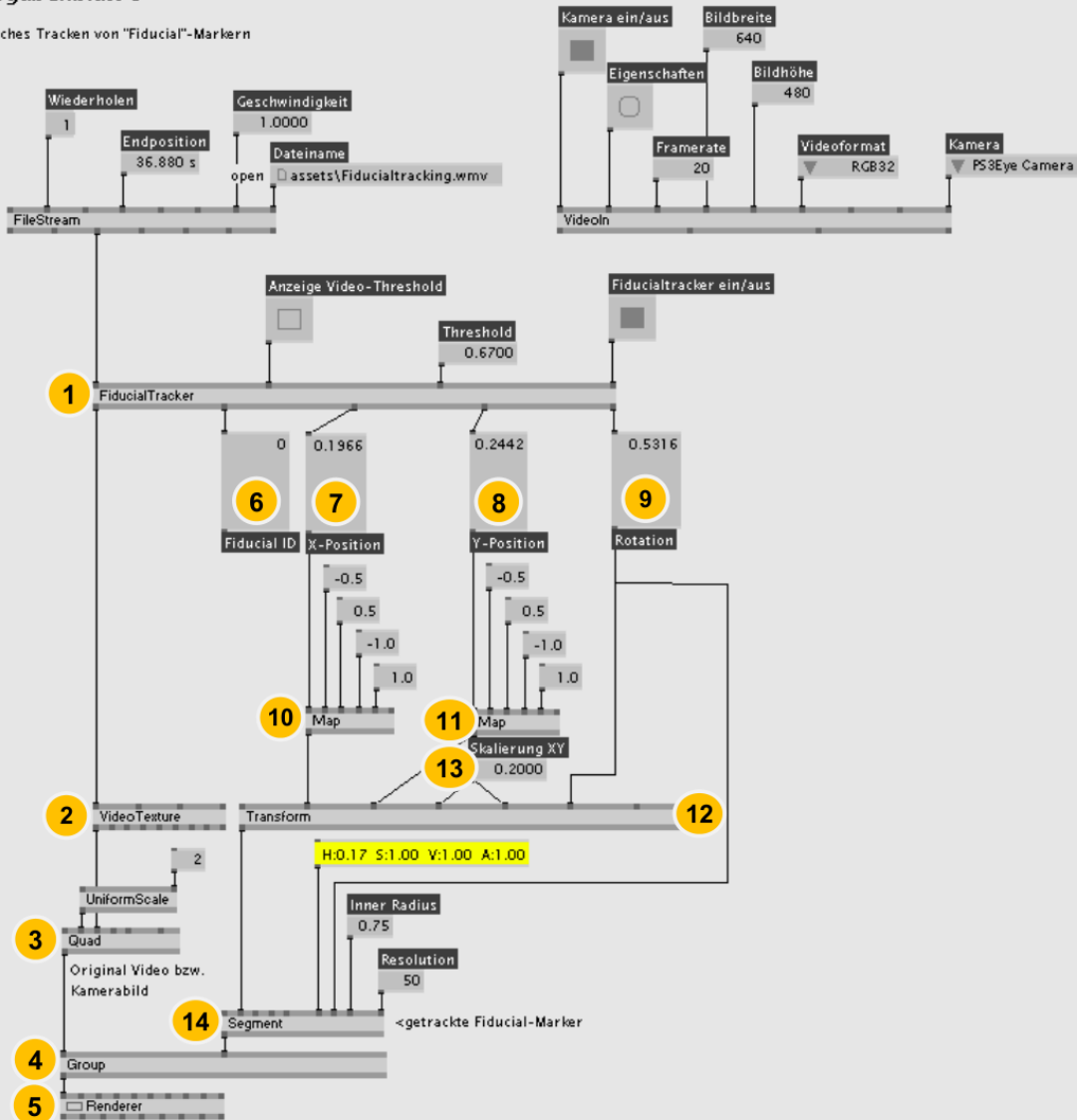


- Wir konfigurieren den „FileStream (DShow9)“-Node wie folgt (bzw. in der Skizze dargestellt):
- Als abzuspielendes Video öffnen wir (mit Rechtsklick auf „Dateiname“) das Video „**Fiducialtracking.wmv**“ im „assets“-Ordner des Projektordners „Fiducialtracking“. Als Abspielgeschwindigkeit wählen wir „1.0“, Wiederholen setzen wir auf „1“ und als Endposition legen wir die Gesamtlänge des Videos (hier: „36.88“ Sekunden) fest.
- Nun bauen wir nach und nach den Patch, wie in der nachfolgenden Skizze dargestellt, auf.
- **Um die Erweiterung dieses Patches zu beschleunigen, finden Sie auf dem Datentausch-Laufwerk „P:“ einen Patch „FiducialTrackingV1_Parametrisierung.v4p“ der bereits die fertige Konfiguration für die Schritte 1 bis 5 sowie 6 bis 14 enthält – und den Sie natürlich verwenden dürfen (copy&paste) ;-). Kopieren Sie also den gesamten Inhalt der Datei in Ihren Patch. Sie sollten dann nur mehr die beiden Knoten „FileStream“ und „Fiducialtracker“ miteinander verbinden müssen.**



Aufgabenblatt 5

Einfaches Tracken von "Fiducial"-Markern



(1) Als erstes fügen wir einen „FiducialTracker (FreeFrame DShow9)“-Node ein und verbinden ihn mit dem Ausgangs-Pin „Video“ des „FileStream (DShow9)“-Nodes. Weiters fügen wir zwei Nodes vom Typ „IOBox (Value Advanced)“ ein (Value Type „Boolean“, Behavior: „Toggle“), eine schließen wir an den Eingangs-Pin „Show Tresholded“ an, die zweite an „Enabled“. Eine weitere „IOBox (Value Advanced)“ (Value Type „Real“) dient als Eingabe für den „Threshold“-Pin.

(2) Einfügen eines „VideoTexture (EX9.Texture VMR9)“-Nodes und verbinden mit dem Ausgangs-Pin „Video Output“ des „FiducialTracker (FreeFrame DShow9)“-Nodes. Diese Komponente ermöglicht es, das Video (bzw. später das Livekamerabild) in einem Renderer-Fenster darzustellen.

(3) bis (5) legt die Konfiguration des Renderer-Fensters fest. Wir fügen zunächst einen „UniformScale (Transform)“-Node ein und legen den Skalierungsfaktor mittels einer „IOBox (Value Advanced)“ auf „2.0“ fest (Value Type: „Real“, Eingangs-Pin „XYZ“; damit wird das Video fensterfüllend dargestellt). Dann fügen wir einen „Quad (DX9)“-Node ein, verbinden den Eingangs-Pin „Texture“ mit dem Ausgangs-Pin „Texture Out“ des „VideoTexture (EX9.Texture VMR9)“-Nodes und den Ausgang des „UniformScale (Transform)“-Nodes mit dem Eingangs-Pin „Transform“ des „Quad (DX9)“-Nodes. In (4) fügen wir einen „Group (EX9)“-Node ein. An den ersten Eingangs-Pin („Layer 1“) wird das Video bzw. Kamerabild gelegt, der zweite Eingangs-Pin wird später die getrackten Objekte darstellen (siehe unten). Weiters fügen wir den schon bekannten „Renderer (EX9)“-Node ein (5) und verbinden ihn mit dem Ausgangs-Pin „Layer“ des „Group (EX9)“-Nodes.

Als Ergebnis sollte nun die Videodatei im Renderer-Fenster abgespielt (bzw. das Livebild der Kamera dargestellt) werden.

Erklärung zu Schritt 6 bis 13 (sollte ebenfalls bereits fertig vorhanden sein): Dieser Teil kümmert sich um die Verarbeitung und Darstellung (im Renderer!) von erkannten Markern im „FiducialTracker (FreeFrame DShow9)“-Node.

(6) bis (9): Wir fügen dazu eine „IOBox (Value Advanced)“ ein und konfigurieren Sie wie folgt: Rows = „5“, Vector Size = „1“, Show Value = „on“. Damit können wir bis zu 5 erkannte Marker gleichzeitig im Patch überwachen. Wir erstellen 3 Kopien dieses Nodes und schließen diese Nodes an die Ausgangs-Pins „Fiducial ID“, „X“, „Y“ und „Orientation“ des „FiducialTracker (FreeFrame DShow9)“-Nodes an.

(10) Wir fügen einen „Map (Value)“-Node ein und verbinden den Ausgangs-Pin „Y Output Value“ der „IOBox (Value Advanced)“ mit der Nummer (7) (=X-Werte der erkannten Marker) mit dessen 1. Eingangs-Pin. Die weitere Konfiguration nehmen wir, so wie auf der Patchskizze dargestellt, vor (Source Minimum=„-0,5“, Source Maximum=„0,5“, Destination Minimum=„-1,0“, Destination Maximum=„1,0“). Dieses Mapping ist notwendig, da der „FiducialTracker“-Node die Position von erkannten Markern im Bereich -0,5...0,5 liefert, das Koordinatensystem im Renderer-Fenster aber von -1,0...1,0 geht.

(11) Ident zu (10) – Sie können eine Kopie erstellen; Der Ausgangs-Pin „Y Output Value“ der „IOBox (Value Advanced)“ mit der Nummer (8) (=Y-Werte der erkannten Marker) wird mit dem 1. Eingangs-Pin dieses Nodes verbunden.

(12) Einfügen eines „Transform (Transform 2d)“-Nodes. Der Ausgabe-Pin „Output“ des ersten (linken) „Map (Value)“-Nodes (10) wird mit dem Eingabe-Pin „TranslateX“ verbunden, der Ausgabe-Pin „Output“ des zweiten (rechten) „Map (Value)“-Nodes (11) wird mit dem Eingabe-Pin „TranslateY“ verbunden und der Ausgabe-Pin „Y Output Value“ der „IOBox (Value Advanced)“ mit der Nummer (9) wird mit dem Eingabe-Pin „Rotate“ des „Transform (Transform 2d)“-Nodes verbunden.

(13) Eine „IOBox (Value Advanced)“ wird mit den Eingangs-Pins „ScaleX“ und „ScaleY“ des „Transform (Transform 2d)“-Nodes verbunden. Mit diesem Wert steuern wir die Größe des/der erkannten Marker(s) im Renderer-Fenster.

(14) Zu guter Letzt fügen wir einen „Segment (DX9)“-Node ein und konfigurieren ihn wie oben dargestellt. Der Ausgangs-Pin „Transform Out“ des „Transform (Transform 2d)“-Nodes geht an den 2. Eingangs-Pin „Transform“ des „Segment (DX9)“-Nodes und der Ausgabe-Pin „Y Output Value“ der „IOBox (Value Advanced)“ mit der Nummer (9) wird mit dem Eingabe-Pin „Cycles“ des „Segment (DX9)“-Nodes verbunden. Weiters definieren wir über eine IOBox (Color) die Farbe des identifizierten Markers (Eingangs-Pin „Color“) sowie über zwei weitere IOBox (Value Advanced)“-Nodes den Innenradius (Anfangswert: „0,75“, Eingangs-Pin „Inner Radius“) sowie die Auflösung (Anfangswert: „50“, Eingangs-Pin „Resolution“).

Damit sind wir mit diesem Patch fertig. Ein Kreisbogen sollte nun bereits dem Marker im Video folgen. Wir können diesen nun im nächsten Schritt mit dem Kamera-Livebild und eigenen Fiducial-Markern testen.

Frage 7: Experimentieren Sie mit dem fertigen Patch und Kamera-Livebild. Was stellen Sie fest, wenn Sie das ganze A4-Blatt mit den 3x4 Fiducial-Markern unter die Kamera halten? Was, wenn Sie das Blatt drehen?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 8: Schneiden Sie nun aus dem A4-Blatt einige Marker aus und kleben Sie diese (z.B. mit Tesa-Klebeband) auf Alltagsgegenstände (z. B. Ihr Smartphone). Was beobachten Sie?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 9: Experimentieren Sie mit dem fertigen Patch und Kamera-Livebild. Was ändert sich bei Veränderung folgender Parameter im Patch (beantworten Sie textuell):

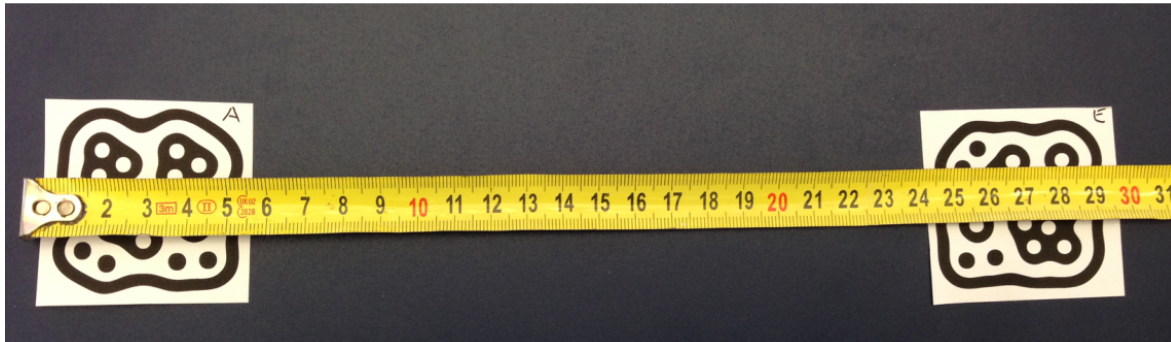
- Threshold-Wert des FiducialTracker-Nodes (in Kombination mit dem Schalter „Anzeige Video-Threshold“)
- „Skalierung XY“ des Transform-Nodes
- „Inner Radius“ bei Segment
- Skalierungswert XYZ des UniformScale-Nodes (Anfangswert: 2)

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Abschließend versuchen wir nun, mit 2 Markern ein einfaches System zum Messen von Abständen, wie nachfolgend dargestellt, zu implementieren. Speichern Sie Ihren Patch zunächst als „FiducialTrackingV2.v4p“ ab. Welche Änderungen müssen Sie vornehmen, damit eine „IOBox (Value Advanced)“ in Ihrem Patch den Abstand zwischen zwei „Fiducial“-Marker in Millimeter oder Zentimeter darstellt/ausgibt?

Hinweis: Zur Abstandsberechnung können Sie auf folgende Formel zurückgreifen; (x_1, y_1) entspricht der Position des 1. Markers, (x_2, y_2) der Position des 2. Markers.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



Frage 10: Dokumentieren Sie die notwendigen Änderungen in Ihrem Patch (textuell, Screenshot) Funktioniert die Abstandsmessung gut? Wie könnte man Sie verbessern? Wo ist der Koordinatenursprung der Marker? Welches Problem erkennen Sie?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

FEEDBACK zu Blatt 6

Bitte füllen Sie den Feedback-Bogen elektronisch in Moodle aus. Sie können diesen aber gerne auch hier abtrennen und anonym abgeben (z.B. zusammengefoldet am Ende eines Praktikumstermins, in der Vorlesung oder in mein Postfach beim I-Dekanat im B-Gebäude, EG). Vielen Dank!

Bitte füllen Sie diese Fragen (**freiwillig**) aus um die Veranstaltung zu verbessern.

- Das behandelte **Thema passt für mich/uns zum Inhalt/Modulbeschreibung der Lehrveranstaltung** (d.h. keine „Themenverfehlung“)...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Die **technische Tiefe/Komplexität** des Aufgabenblattes war **zu hoch**...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Dieses Aufgabenblatt hat mir/hat uns **Spaß** gemacht...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Ich/wir haben etwas **gelernt**, möchten das **in Zukunft evtl. selbst anwenden**...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Die Aufgabenstellungen waren **einfach genug** um von mir/uns **verstanden** zu werden...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Die **Zusatz-/Hintergrundinformationen** auf dem Aufgabenblatt waren für mich/uns **nützlich**...

☐ Stimme voll zu ☐ Stimme zu ☐ Ausgewogen ☐ Eher nicht ☐ Absolut nicht

- Verbesserungsmöglichkeiten? ...können Sie in diesem Feld angeben falls Sie möchten

Feedback/Kritik wird selbstverständlich vertraulich behandelt und in keiner Weise zur Bewertung der Leistung des Studenten/der Gruppe herangezogen!